Centinela API

Centinela se integra con tus circuitos en dos niveles:

1. **Biblioteca Python** (`ddqpu.Centinela`): corre en tu máquina o en Kaggle, usa tu cuenta de IBM y
   devuelve circuitos ISA listos para `SamplerV2`.
2. **Servidor REST** (`ddqpu.servidor`): opcional. Guarda la bitácora compartida y hace el análisis;
   nunca recibe tu token de IBM, solo conteos y metadatos.

Inicio rápido

```
from qiskit_ibm_runtime import QiskitRuntimeService
from ddqpu import Centinela

backend = QiskitRuntimeService().backend("ibm_fez")
cen = Centinela(backend, bitacora="centinela/bitacora.jsonl", max_qpu_s=30)

print(cen.estimar_costo(mis_circuitos, shots=4000))   # no envía nada
prep = cen.preparar(mis_circuitos)                    # sondeo + layout + DD + fase
print(prep)                                           # qué decidió y por qué
ej = cen.ejecutar(prep, shots=4000)                   # tus PUBs + testigo al final
print(ej.etiqueta, ej.counts(0))
```

Si ya tienes tu propio flujo de ejecución, usa solo `preparar` y envía `prep.pubs(shots)` con tu `SamplerV2`, `Session` o `Batch`.

Referencia

`Centinela(backend, bitacora=..., motor=None, mode=None, probe_config=None, max_edad_s=300, max_qpu_s=30, max_sondeo=12, umbral_ganancia_dd=1.15, penal_degradado=0.25, penal_sin_sondear=0.02, sampler_options=None, ejecutor=None)`

Parámetro	Uso
`backend`	`IBMBackend` o backend fake (`FakeFez`); en simulación usa `backend_simulable(FakeFez())`
`bitacora`	archivo JSONL con el historial por qubit (la huella de referencia sale de aquí)
`motor`	`MotorLocal` (por defecto) o `MotorRemoto(url, api_key)`
`mode`	`Session` o `Batch` para `SamplerV2`; por defecto, el backend
`probe_config`	`ProbeConfig()` estándar (28 circuitos por grupo) o `SONDEO_LIGERO` (sin T1, ~40 % más barato)

Parámetro	Uso
`max_edad_s`	un sondeo más joven que esto se reutiliza
`max_qpu_s`	freno: si el sondeo estimado excede esto, no se envía
`max_sondeo`	máximo de qubits a sondear en modo automático (los del circuito + vecinos)
`ejecutor`	callable `(pubs) -> (job_id, [counts])` para simuladores propios o pruebas (`EjecutorSintetico`)

Métodos

Método	Devuelve	Notas
`estimar_costo(circuitos, shots)`	dict	segundos y USD a precio de lista del sondeo, testigo y tus circuitos, y el sobrecosto en %
`sondear(qubits=None, k=4, incluir=(), forzar=False)`	`Informe`	un job; grupos sin vecinos directos
`preparar(circuitos, informe=None, dd="auto", compensar_fase=True, evitar_degradados=True, optimization_level=2, **transpile_kw)`	`Preparado`	con `initial_layout=[...]` respeta tu layout y solo aplica DD y fase
`ejecutar(prep_o_circuitos, shots, testigo="auto")`	`Ejecucion`	`testigo`: `"auto"`, `"post"`, `"ambos"` o `False`
`elegir_layout(circuitos, informe, duraciones_s, restringir_a=None)`	dict o `None`	el selector de qubits, expuesto por si quieres usarlo solo
`target_penalizado(informe)`	`Target`	target con error extra en qubits degradados, para usarlo con tu propio transpilador

Objetos

- `Informe`: `medidas[q]` (T2*, T2 eco, T1, detuning, Гф, errores de lectura), `veredictos[q]` (estado, motivo, ganancia DD, referencia), `degradados`, `estables`, `guardar()`, `from_dict()`.
- `Preparado`: `circuitos` (ISA), `qubits`, `dd_qubits`, `compensacion_kHz`, `decisiones`, `advertencias`, `pubs(shots)`.
- `Ejecucion`: `resultados` (PubResult), `counts(i)`, `testigo` (`pre`/`post` por qubit), `etiqueta`, `etiquetas_qubit`, `costo_testigo`.

Etiquetas: `estable`, `degradado`, `cambio_durante_job` (el testigo detectó un salto de frecuencia de más de 1.5 kHz o un cambio de estado) y `sin_datos`.

Cómo decide

1. **Qubits candidatos**: los que asignaría Qiskit a tu circuito más sus vecinos, hasta `max_sondeo`.
2. **Grupos**: dos qubits vecinos nunca se sondean en el mismo circuito. El acoplamiento ZZ (−4 kHz en q142) sesgaría el detuning y T2*.
3. **Estado**: `degradado` si T2* + 2σ < 0.6 × referencia, o si el detuning se aleja de la referencia más de max(1 kHz, 3σ).
4. **Layout**: enumera los embebidos sin SWAP (VF2), primero solo entre los qubits estables sondeados, y minimiza la suma, sobre todos tus circuitos, de −ln(1 − e) con e = error de compuerta 1q y 2q, de lectura y de espera, donde p_espera = (1 − e^(−D/T2*))/2 usa el T2* medido ahora. Si no hay un layout estable, sondea los qubits del mejor layout del chip antes de aceptarlo (sondeo adaptativo).
5. **DD**: X-X (nativo en IBM) solo en qubits degradados, no sondeados o con eco/T2* ≥ 1.15.
6. **Fase**: `rz(−2π f τ)` después de cada `delay` si |f| > max(3σ, 0.5 kHz).

Límites conocidos

- La corrección de fase cubre el detuning estático con los vecinos en |0⟩. El corrimiento ZZ con vecinos excitados depende de tu circuito y no se corrige.
- La fase acumulada durante las compuertas no se compensa (menos de 0.015 rad a 27 kHz).
- Los testigos asumen que los PUBs de un job se ejecutan en orden.
- El costo fijo (unos 7–12 s de sondeo y 3–6 s de testigo) hace que Centinela convenga en trabajos de minutos, no de segundos. `estimar_costo` lo reporta antes de gastar.
- En simulación (Aer) no hay TLS ni ruido 1/f, así que Centinela y la transpilación estándar dan resultados equivalentes. El valor solo se puede medir en hardware (sección 7 del notebook).

Servidor REST

```
pip install "ddqpu[api]"
CENTINELA_BITACORA=/datos/bitacora.jsonl CENTINELA_API_KEY=secreto \
  uvicorn ddqpu.servidor:app --host 0.0.0.0 --port 8000
# o con Docker
docker build -t centinela . && docker run -p 8000:8000 -e CENTINELA_API_KEY=secreto
```

Método	Ruta	Cuerpo / respuesta
GET	`/v1/salud`	`{"ok": true, "version": ...}`
POST	`/v1/sondeos`	`{backend, manifest: {specs, candidates}, counts, T2_cal_us?, guardar?, extra?}` → `{medidas, veredictos}`

Método	Ruta	Cuerpo / respuesta
POST	`/v1/evaluar`	`{backend, medidas: {q: {...}}}` → `{veredictos}`
GET	`/v1/backends/{backend}/qubits/{q}`	referencia, último registro, `n_sondeos`, `fraccion_degradado`
GET	`/v1/backends/{backend}/qubits/{q}/historial?limite=100`	`{historial: [...]}`

Autenticación: cabecera `X-API-Key` cuando `CENTINELA_API_KEY` está definida. La documentación interactiva
(OpenAPI) queda en `/docs`.

Cliente:

```
from ddqpu import Centinela, MotorRemoto
cen = Centinela(backend, motor=MotorRemoto("https://centinela.tu-dominio", api_key="s
```

Antes de operar una bitácora compartida entre terceros, revisa los términos de uso de IBM Quantum
sobre la redistribución de datos de caracterización.